Prefix Caching KV Router

Technical project brief for software engineering internships

Brad Asher | Rust | github.com/BradshawAsher/prefix-caching-proxy

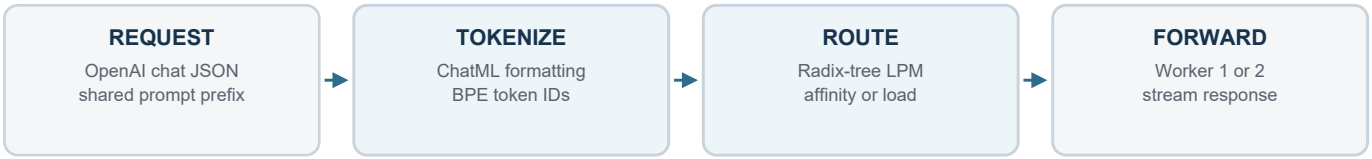
Project overview

I built an asynchronous Layer 7 reverse proxy in Rust to preserve prompt-cache locality across distributed LLM inference workers. Traditional round-robin balancing can send related requests to different workers, forcing each worker to repeat the prompt prefill stage even when another worker already holds the matching key-value cache.

The proxy accepts OpenAI-compatible chat requests, formats and tokenizes each prompt, finds the longest cached token prefix in a compressed radix tree, and routes to the matching worker when at least 20 percent of the prompt is reusable. Weak matches and misses fall back to the worker with the fewest reported requests. Downstream Server-Sent Events are passed through as an asynchronous byte stream so the proxy does not buffer a complete model response before returning output.

Rust 2024 Axum and Tokio	**6 of 6** tests passing	**20 percent** affinity threshold	**2 workers** configurable targets

Request path



After a successful downstream response, the proxy registers the prompt-worker mapping in the radix tree.

Core design

COMPONENT	IMPLEMENTATION AND PURPOSE
Prompt pipeline	Chat messages are serialized in a consistent ChatML-style order and encoded with a local Hugging Face BPE tokenizer. Matching token IDs mirrors the sequence used by the model more closely than comparing raw text.
Compressed radix tree	Each edge stores a contiguous token slice. Insertions split partially matching branches, while longest-prefix lookup credits workers at every matched depth and resolves ties deterministically.
Routing policy	A prefix match at or above the 20 percent threshold receives cache affinity. Otherwise, atomic request counters provide a least-loaded fallback instead of forcing a weak cache match.
Proxy and observability	Axum handles health, statistics, and chat endpoints. A pooled reqwest client forwards JSON, preserves the downstream status and content type, and streams SSE bodies without full-response buffering.

Engineering decisions and tradeoffs

I treated the router as a systems prototype and kept the affinity policy observable. The current design makes cache reuse easy to reason about, while its limitations define clear production work.

DECISION	WHY IT WORKS	CURRENT COST
Match token IDs	Routing uses the model-facing prompt sequence and can recognize a reusable partial prefix.	The proxy must use the same tokenizer and prompt template as the downstream model.
Compress radix edges	Shared paths store token runs instead of allocating one node per token, and branch splitting preserves diverging prompts.	The tree is process-local and has timestamps but no implemented eviction or worker-invalidation policy.
Use shared reads and deferred inserts	Longest-prefix queries take a read lock; successful requests register their prompt in a spawned task after the downstream response arrives.	One global write lock still protects all mutations, and the asynchronous update is not persisted across restarts.
Combine affinity with load fallback	A minimum match ratio avoids routing weak matches solely for locality, while misses choose the lowest counter.	A strong match overrides queue depth. The current counter is decremented when response headers arrive, so it does not cover the full SSE stream lifetime.

Verification

The Rust test suite passes 6 of 6 tests. Radix-tree tests cover exact matches, partial matches, dynamic branch splitting across workers, common-prefix queries, and complete misses. Tokenizer tests cover chat formatting and verify that an extended prompt preserves the shorter prompt's token prefix. The suite does not yet exercise the HTTP proxy end to end or simulate concurrent failures.

Benchmark harness status

A local harness sent 15 sequential requests with one shared system prompt and different user questions to two Python mock workers. The workers simulated 175 ms cold-prefill and 20 ms warm-cache delays; a request below 50 ms was counted as a cache hit. The round-robin phase ran first without resetting worker caches, so the figures below are observed values from the saved run, not a controlled estimate of the router's causal benefit.

METRIC	ROUND ROBIN	PREFIX ROUTER	OBSERVED CHANGE
Cache hit rate	86.7 percent	100.0 percent	+13.3 percentage points
Median TTFT	24.2 ms	29.3 ms	+5.1 ms
P95 TTFT	287.8 ms	33.7 ms	-254.1 ms or 88.3 percent

Next engineering steps

The next version would keep each active-request guard alive until its response stream closes, add worker health checks and retry behavior, and expire prefixes when a worker evicts its KV cache. I would also reset worker state between benchmark scenarios, then run concurrent integration tests against a real inference server and measure routing CPU time, throughput, and TTFT distributions under sustained load.

What this project demonstrates

This project gave me hands-on experience with asynchronous networking, compact indexing structures, concurrency primitives, streaming HTTP, and performance measurement. It also required separating cache-locality gains from proxy overhead and documenting where a controlled simulation stops representing production inference.